

get_version

REFERENCE

API consumer

read the reader's identity and software versions

Overview

`get_version` retrieves the reader's **identity and software-version metadata** in a single round trip: model, serial number, SKU, top-level firmware version, and a component-level breakdown (scanner, radio, and IoT Connector versions). It is a read-only query on the device-management ([MGMT](#)) endpoint — the app publishes the command and the device publishes a correlated reply containing the `readerVersion` object.

Because it is idempotent, read-only, and independent of radio state, `get_version` can be issued **at any point once the connection is active**, and it is the natural first command to send after any connect, reconnect, or bootstrap — to confirm which device you are talking to and which command set it supports. Where `get_version` reports *identity*, `get_status` reports *live runtime health*.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	v1.21 (API Version Reporting)
Related	<code>get_status</code> , <code>get_current_region</code> , <code>set_os</code>

When to Use

Use `get_version` when you need to **establish device identity and the software baseline** — typically in a management or fleet workflow over the MGMT endpoint. It answers "is this the

device I expect, and what command set does it support?"

- **Identify the device:** `readerVersion.model`, `serialNumber`, and `sku` confirm the exact model and regional/hardware variant, and key asset-tracking, support tickets, and device registration to the physical label.
- **Gate on firmware / capabilities:** compare `readerVersion.firmwareVersion` and `detailedVersions` against your expected baseline before an update or when diagnosing an issue; read `detailedVersions.iotcVersion` to decide which MQTT API commands and features are available.
- **Confirm a firmware update applied:** `set os` returns only an *accepted* acknowledgment (`code 1`) with no terminal success/failure on the response topic — poll `get_version.readerVersion.firmwareVersion` to verify the new firmware actually took effect.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `readerVersion.model` (response) — `RFD40` or `RFD90`; branch model-specific configuration on it.
- `readerVersion.firmwareVersion` (response) — the reader firmware build; compare against your baseline and use it to confirm a `set os` update applied.
- `readerVersion.detailedVersions.iotcVersion` (response) — the IoT version, which determines the available command/feature set.
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_version" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_version	Fixed command name. Retrieves the reader's identity and version information.
2	requestId *	string	—	abc123	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command`, `requestId`, `apiVersion`, `response`, and a `readerVersion` object. On the **Schema** tab, expand `readerVersion` (and its `detailedVersions` sub-object) to see every field — its type, whether it is required, its enum/units, an example value, and a description; the **Example** tab shows the full JSON.

`readerVersion` is optional at the envelope level, but when it is present its six identity members (`firmwareVersion`, `model`, `serialNumber`, `sku`, `companyName`, `manufacturerName`) are all required; `detailedVersions` and its members are optional.

Response payload

```
1      {
2          "command" : "get_version" ,
3          "requestId" : "abc123" ,
4          "apiVersion" : "V1.21" ,
5          "readerVersion" : {
6              "firmwareVersion" : "PAAFKS00-013-N05" ,
7              "model" : "RFD40" ,
8              "serialNumber" : "211545201D0034" ,
9              "sku" : "RFD4031-G10B700-US" ,
10             "companyName" : "Zebra Technologies" ,
11             "manufacturerName" : "Zebra Technologies" ,
12             "detailedVersions" : {
13                 "scannerFirmware" : "PAAEOC20-003-R01" ,
14                 "radioFirmware" : "2.0.53.0" ,
15                 "iotcVersion" : "V1.22"
16             }
17         } ,
18         "response" : {
19             "code" : 0 ,
20             "description" : "Success"
21         }
22     }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_version	Echoes the executed command.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abcd123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API/IoTC protocol version reported by this command. See the caveat in Response Semantics — it is emitted per-handler, not device-wide.
4	readerVersion	object	—	—	Reader identity and software-version details. Optional at the envelope level; when present, all six identity members below are required.
5	firmwareVersion *	string	—	PAAFKS00-006-R02	The firmware version of the reader.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
6	model	enum	RFD40 RFD90	RFD40	The reader model.
7	serialNumber	string	—	23053520102096	The reader serial number (matches the device label).
8	sku	string	—	RFD4031-G10B700-US	The stock-keeping unit — identifies the regional / hardware variant.
9	companyName	enum	Zebra Technologies	Zebra Technologies	The company name associated with the reader.
10	manufacturerName	enum	Zebra Technologies	Zebra Technologies	The manufacturer name associated with the reader.
11	details	object	—	—	Component-level version breakdown. Optional.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
12	s c a n n e r F i r m w a r e	string	—	PAAEOC20-003- R01	Firmware version of the scanner component.
13	r a d i o F i r m w a r e	string	—	2.0.42.0	Firmware version of the RFID radio component.
14	i o t c v e r s i o n	enum	V1.1 V1.2 V1.21	V1.1	The version of the IoT Connector (IoTc) software.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
15	response *	object	—	—	Result wrapper.
16	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
17	description *	string	—	Success	Human-readable status.

Response Semantics

- **Version fields.** Use `readerVersion.detailedVersions.iotcVersion` as the IoT C software version, and compare firmware builds via `readerVersion.firmwareVersion`. The envelope `apiVersion` is per-handler — see [API Version Reporting](#).
- `iotcVersion` **can be newer than its enum.** The contract's example reports `v1.22`, but the field's enum lists only `v1.1`, `v1.2` and `v1.21`, so the contract's own example does not validate against it — a known issue in the source schema. Parse `iotcVersion` as a string and compare versions numerically rather than rejecting an unlisted value.
- `companyName` and `manufacturerName` are constrained to `Zebra Technologies`. `readerVersion.detailedVersions` is optional and may be absent on some builds; the example values shown (firmware, SKU, serial) are illustrative — real values vary by unit and region.

Response Codes

`get_version` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — identity and version details retrieved and returned.
101	Default error — generic catch-all.

Examples

`get_version` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abc123`) is echoed back so you can pair reply to request. The full reply, field by field, is shown under [Response Payload](#) above.

Notes and Caveats

- Confirming a firmware update: `set_os` returns an **acceptance response only** — `code 1` ("Command payload is accepted") on the response topic, with no terminal success or failure. While the update runs, the reader emits live progress events: a `FIRMWARE_UPDATE` `alert` carrying `alertDetails.fwUpdateStatus` (`updateStatus` , `overallProgress` , `stage`), provided firmware-update events are enabled with `config_events` (`eventConfiguration.firmwareUpdate: true`). To confirm the update applied, watch those events, or poll `get_version` and check that `readerVersion.firmwareVersion` advanced to the expected build.

`get_version` is idempotent and independent of radio state, so it can be issued at any time on an active connection and is the right **first command after any connect, reconnect, or bootstrap** — confirm the device identity and IoT version before relying on newer commands or fields. See [Management](#).

Related:  [Management](#) ·  `get_status` ·  `set_os` ·  [Response Codes](#)